Breast Cancer Detection using Decision Tree and Random Forest

Siddhesh

2026-10-23

Contents

1. Data Collection **1**

2. Data Pre-processing and Wrangling **3**

3. Exploratory and Inferential Analysis **4**

3.1 Distribution of Diagnoses 4

3.2 Compare Mean Radius 5

3.3 Correlation Analysis 6

3.4 Statistical Test 7

4. Model Training **8**

4.1 Decision Tree 8

5. Model Testing and Evaluation **9**

5.1 ROC Curves 10

6. Feature Importance **11**

7. Conclusion and Limitations **12**

1. Data Collection

This project uses the Breast Cancer Wisconsin Diagnostic dataset. It contains measurements computed from digitized images of breast mass samples. The objective is to compare a Decision Tree and a Random Forest model for classifying the diagnosis as benign or malignant.

```
packages <- c("tidyverse", "caret", "rpart", "rpart.plot",
              "randomForest", "corrplot", "pROC")

missing_packages <- packages[
  !vapply(packages, requireNamespace, logical(1), quietly=TRUE)
]
```

```r
if (length(missing_packages) > 0) {
  install.packages(missing_packages, repos="https://cloud.r-project.org")
}

invisible(lapply(packages, library, character.only=TRUE))
```

```r
raw <- read.csv("data.csv", check.names=TRUE)

cat("Dataset dimensions:", dim(raw), "\n")
```

```
## Dataset dimensions: 569 33
```

```r
head(raw)
```

```
##         id diagnosis radius_mean texture_mean perimeter_mean area_mean
## 1   842302         M       17.99        10.38         122.80    1001.0
## 2   842517         M       20.57        17.77         132.90    1326.0
## 3 84300903         M       19.69        21.25         130.00    1203.0
## 4 84348301         M       11.42        20.38          77.58     386.1
## 5 84358402         M       20.29        14.34         135.10    1297.0
## 6   843786         M       12.45        15.70          82.57     477.1
##   smoothness_mean compactness_mean concavity_mean concave.points_mean
## 1         0.11840          0.27760         0.3001             0.14710
## 2         0.08474          0.07864         0.0869             0.07017
## 3         0.10960          0.15990         0.1974             0.12790
## 4         0.14250          0.28390         0.2414             0.10520
## 5         0.10030          0.13280         0.1980             0.10430
## 6         0.12780          0.17000         0.1578             0.08089
##   symmetry_mean fractal_dimension_mean radius_se texture_se perimeter_se
## 1        0.2419                0.07871    1.0950     0.9053        8.589
## 2        0.1812                0.05667    0.5435     0.7339        3.398
## 3        0.2069                0.05999    0.7456     0.7869        4.585
## 4        0.2597                0.09744    0.4956     1.1560        3.445
## 5        0.1809                0.05883    0.7572     0.7813        5.438
## 6        0.2087                0.07613    0.3345     0.8902        2.217
##   area_se smoothness_se compactness_se concavity_se concave.points_se
## 1  153.40      0.006399        0.04904      0.05373           0.01587
## 2   74.08      0.005225        0.01308      0.01860           0.01340
## 3   94.03      0.006150        0.04006      0.03832           0.02058
## 4   27.23      0.009110        0.07458      0.05661           0.01867
## 5   94.44      0.011490        0.02461      0.05688           0.01885
## 6   27.19      0.007510        0.03345      0.03672           0.01137
##   symmetry_se fractal_dimension_se radius_worst texture_worst perimeter_worst
## 1     0.03003             0.006193        25.38         17.33          184.60
## 2     0.01389             0.003532        24.99         23.41          158.80
## 3     0.02250             0.004571        23.57         25.53          152.50
## 4     0.05963             0.009208        14.91         26.50           98.87
## 5     0.01756             0.005115        22.54         16.67          152.20
## 6     0.02165             0.005082        15.47         23.75          103.40
##   area_worst smoothness_worst compactness_worst concavity_worst
## 1     2019.0           0.1622            0.6656          0.7119
## 2     1956.0           0.1238            0.1866          0.2416
```

```
## 3     1709.0           0.1444            0.4245          0.4504
## 4      567.7           0.2098            0.8663          0.6869
## 5     1575.0           0.1374            0.2050          0.4000
## 6      741.6           0.1791            0.5249          0.5355
##   concave.points_worst symmetry_worst fractal_dimension_worst  X
## 1               0.2654         0.4601                 0.11890 NA
## 2               0.1860         0.2750                 0.08902 NA
## 3               0.2430         0.3613                 0.08758 NA
## 4               0.2575         0.6638                 0.17300 NA
## 5               0.1625         0.2364                 0.07678 NA
## 6               0.1741         0.3985                 0.12440 NA
```

2. Data Pre-processing and Wrangling

We remove identifier columns, check for missing values, and convert the diagnosis into a factor for classification.

```
names(raw) <- trimws(names(raw))

if (!"diagnosis" %in% names(raw)) {
  stop("The dataset has no 'diagnosis' column. Check data.csv.")
}

df <- raw %>%
  select(-any_of(c("id", "X", "Unnamed: 32"))) %>%
  mutate(
    diagnosis = factor(
      diagnosis,
      levels=c("B", "M"),
      labels=c("Benign", "Malignant")
    )
  ) %>%
  filter(!is.na(diagnosis))

cat("Rows and columns after cleaning:", dim(df), "\n")
```

```
## Rows and columns after cleaning: 569 31
```

```
cat("Missing values:", sum(is.na(df)), "\n")
```

```
## Missing values: 0
```

```
table(df$diagnosis)
```

```
##
##    Benign Malignant
##       357       212
```

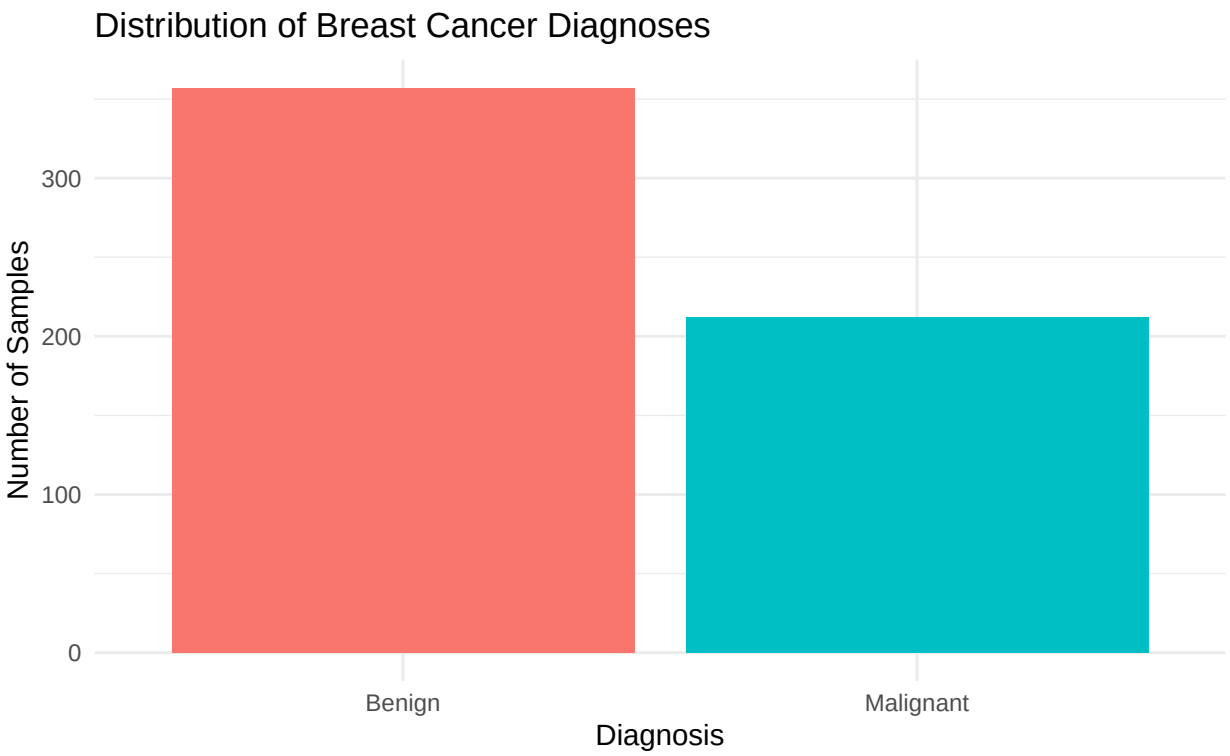
```r
missing_summary <- colSums(is.na(df))
data.frame(
  Feature=names(missing_summary),
  MissingValues=as.integer(missing_summary)
)
```

```
##                    Feature MissingValues
## 1                diagnosis             0
## 2              radius_mean             0
## 3             texture_mean             0
## 4           perimeter_mean             0
## 5                area_mean             0
## 6          smoothness_mean             0
## 7         compactness_mean             0
## 8           concavity_mean             0
## 9       concave.points_mean             0
## 10           symmetry_mean             0
## 11  fractal_dimension_mean             0
## 12                radius_se             0
## 13               texture_se             0
## 14             perimeter_se             0
## 15                  area_se             0
## 16            smoothness_se             0
## 17           compactness_se             0
## 18             concavity_se             0
## 19        concave.points_se             0
## 20              symmetry_se             0
## 21     fractal_dimension_se             0
## 22             radius_worst             0
## 23            texture_worst             0
## 24          perimeter_worst             0
## 25               area_worst             0
## 26         smoothness_worst             0
## 27        compactness_worst             0
## 28          concavity_worst             0
## 29     concave.points_worst             0
## 30           symmetry_worst             0
## 31 fractal_dimension_worst             0
```

3. Exploratory and Inferential Analysis

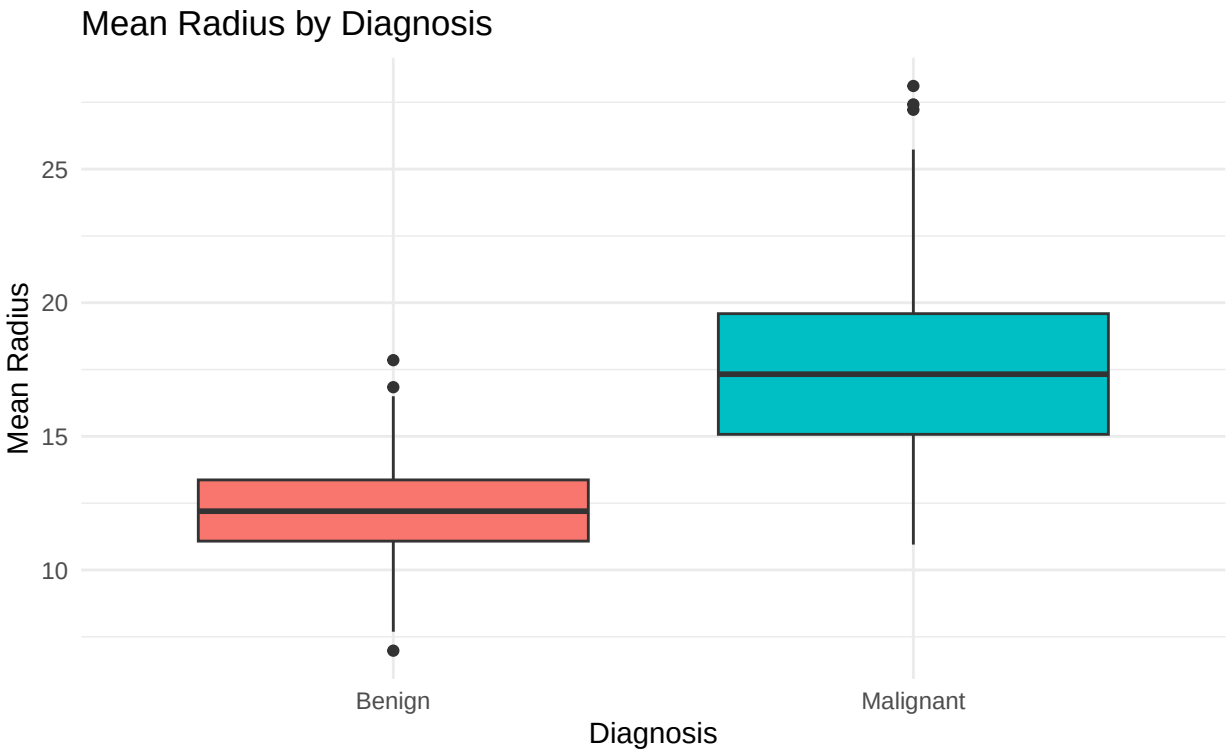
3.1 Distribution of Diagnoses

```r
ggplot(df, aes(x=diagnosis, fill=diagnosis)) +
  geom_bar() +
  labs(title="Distribution of Breast Cancer Diagnoses",
       x="Diagnosis", y="Number of Samples") +
  theme_minimal() +
  theme(legend.position="none")
```



3.2 Compare Mean Radius

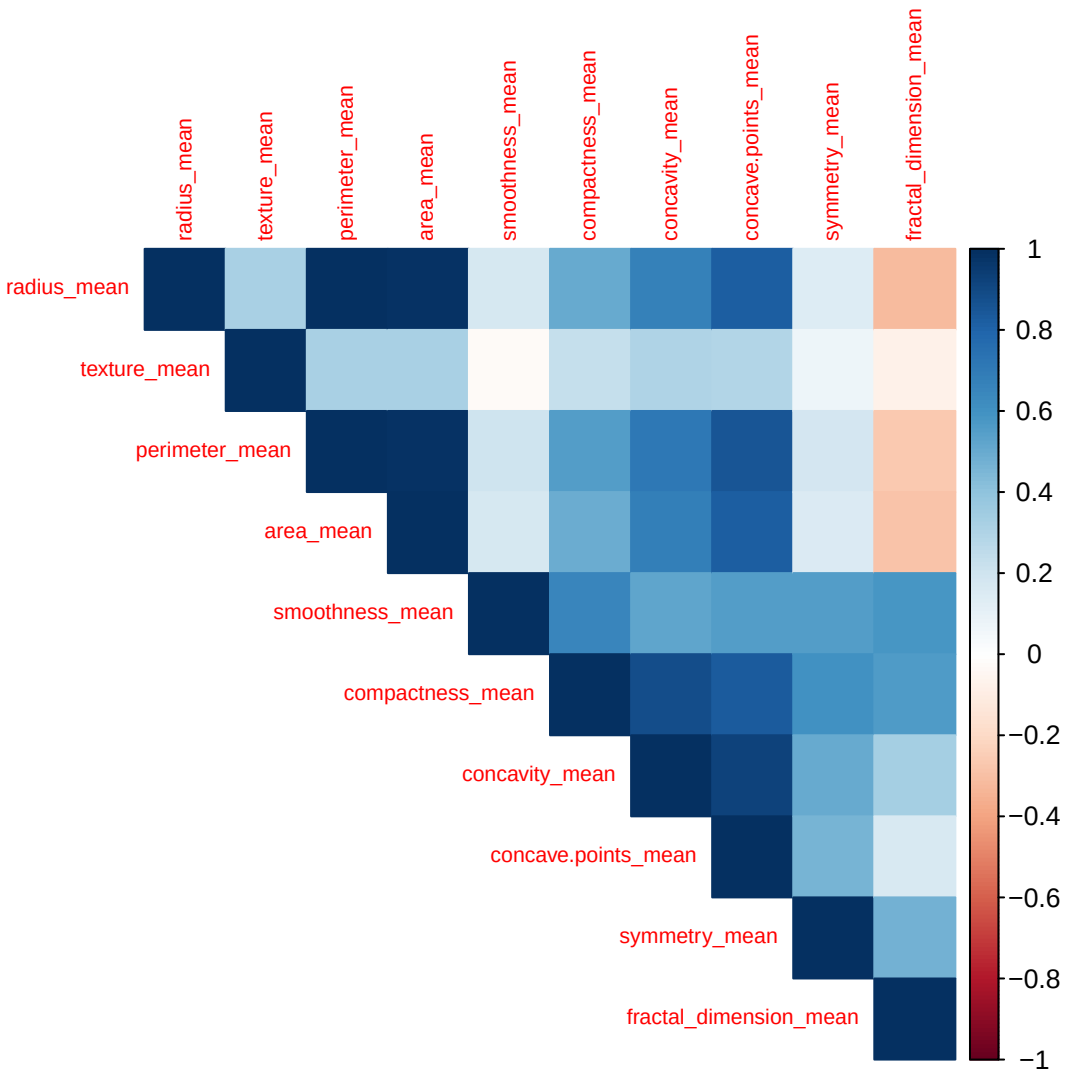
```
ggplot(df, aes(x=diagnosis, y=radius_mean, fill=diagnosis)) +
  geom_boxplot() +
  labs(title="Mean Radius by Diagnosis",
       x="Diagnosis", y="Mean Radius") +
  theme_minimal() +
  theme(legend.position="none")
```



3.3 Correlation Analysis

```
mean_features <- df %>% select(ends_with("_mean"))
cor_matrix <- cor(mean_features, use="complete.obs")

corrplot(cor_matrix, method="color", type="upper",
         tl.cex=0.7, mar=c(0,0,2,0),
         title="Correlation Between Mean Features")
```



3.4 Statistical Test

We use a Welch two-sample t-test to compare mean radius between the two diagnosis groups.

```
radius_test <- t.test(radius_mean ~ diagnosis, data=df)
radius_test
```

```
##
##  Welch Two Sample t-test
##
## data:  radius_mean by diagnosis
## t = -22.209, df = 289.71, p-value < 2.2e-16
## alternative hypothesis: true difference in means between group Benign and group Malignant is not equa
## 95 percent confidence interval:
##  -5.787448 -4.845165
```

```
## sample estimates:
##    mean in group Benign mean in group Malignant
##                12.14652                 17.46283
```

4. Model Training

We split the dataset into training and testing sets. The models learn from the training data and are evaluated on held-out test data.

```
idx <- createDataPartition(df$diagnosis, p=0.7, list=FALSE)
train <- df[idx, ]
test <- df[-idx, ]

tree <- rpart(diagnosis ~ ., data=train, method="class")

rf <- randomForest(
  diagnosis ~ ., data=train,
  ntree=500, importance=TRUE
)

cat("Training samples:", nrow(train), "\n")
```

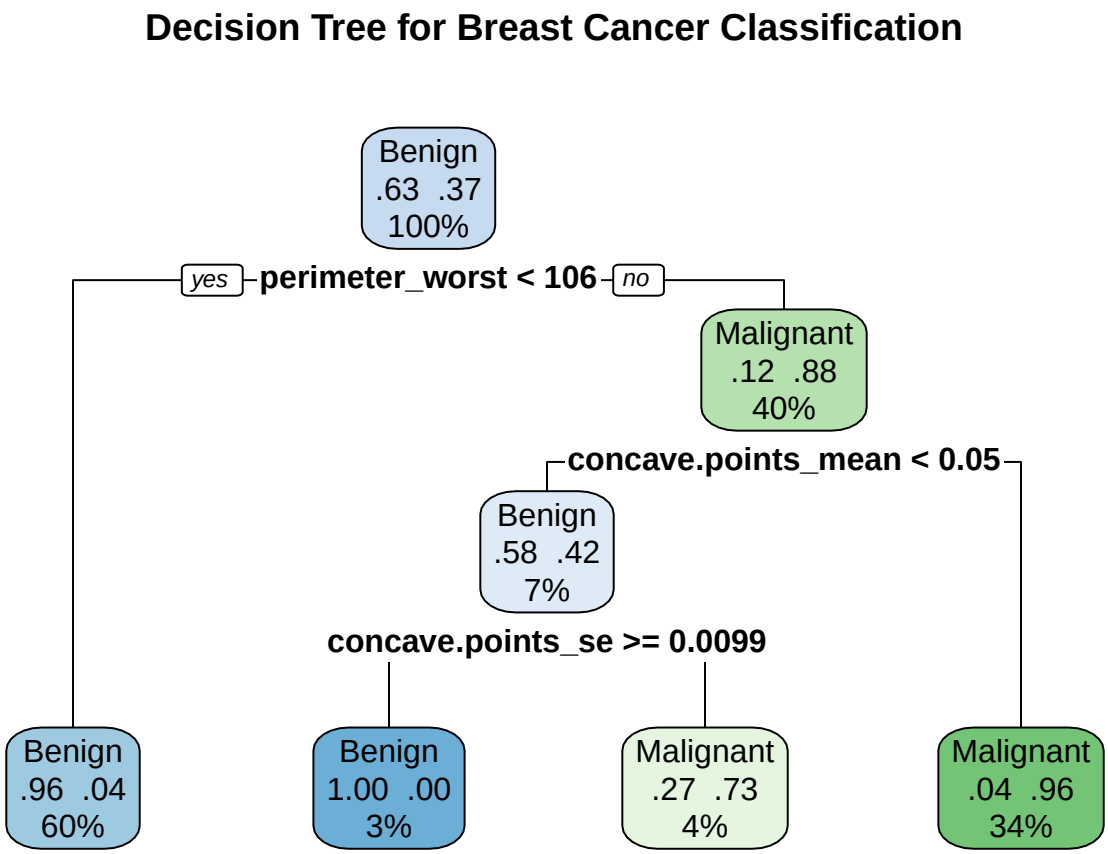
```
## Training samples: 399
```

```
cat("Testing samples:", nrow(test), "\n")
```

```
## Testing samples: 170
```

4.1 Decision Tree

```
rpart.plot(tree, extra=104,
           main="Decision Tree for Breast Cancer Classification")
```



5. Model Testing and Evaluation

We evaluate both models using a confusion matrix, accuracy, sensitivity, and specificity. These are educational model evaluation metrics, not clinical validation.

```
tree_pred <- predict(tree, test, type="class")
rf_pred <- predict(rf, test, type="class")

tc <- confusionMatrix(
  data=tree_pred, reference=test$diagnosis,
  positive="Malignant"
)

rc <- confusionMatrix(
  data=rf_pred, reference=test$diagnosis,
  positive="Malignant"
)

cat("Decision Tree confusion matrix:\n")
```

```
## Decision Tree confusion matrix:
```

```r
print(tc$table)
```

```
##            Reference
## Prediction  Benign Malignant
##   Benign       101        10
##   Malignant      6        53
```

```r
cat("Random Forest confusion matrix:\n")
```

```
## Random Forest confusion matrix:
```

```r
print(rc$table)
```

```
##            Reference
## Prediction  Benign Malignant
##   Benign       104         4
##   Malignant      3        59
```

```r
metrics <- data.frame(
  Model=c("Decision Tree", "Random Forest"),
  Accuracy=c(
    unname(tc$overall["Accuracy"]),
    unname(rc$overall["Accuracy"])
  ),
  Sensitivity=c(
    unname(tc$byClass["Sensitivity"]),
    unname(rc$byClass["Sensitivity"])
  ),
  Specificity=c(
    unname(tc$byClass["Specificity"]),
    unname(rc$byClass["Specificity"])
  )
)

knitr::kable(metrics, digits=3,
             caption="Comparison of Model Performance")
```

Table 1: Comparison of Model Performance

Model	Accuracy	Sensitivity	Specificity
Decision Tree	0.906	0.841	0.944
Random Forest	0.959	0.937	0.972

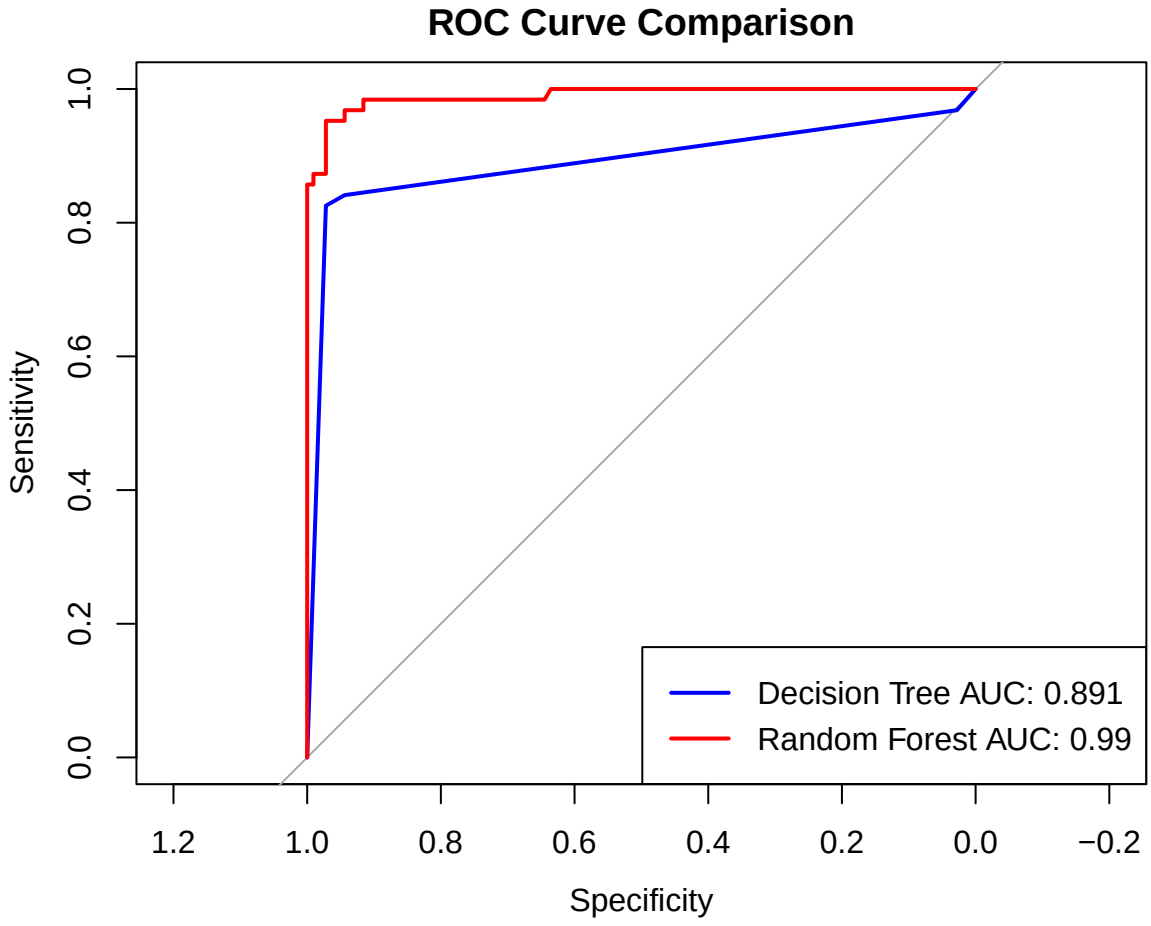
5.1 ROC Curves

```r
tree_prob <- predict(tree, test, type="prob")[, "Malignant"]
rf_prob <- predict(rf, test, type="prob")[, "Malignant"]
```

```r
r1 <- roc(test$diagnosis, tree_prob,
          levels=c("Benign", "Malignant"), quiet=TRUE)
r2 <- roc(test$diagnosis, rf_prob,
          levels=c("Benign", "Malignant"), quiet=TRUE)

plot(r1, col="blue", lwd=2,
     main="ROC Curve Comparison")
plot(r2, col="red", lwd=2, add=TRUE)

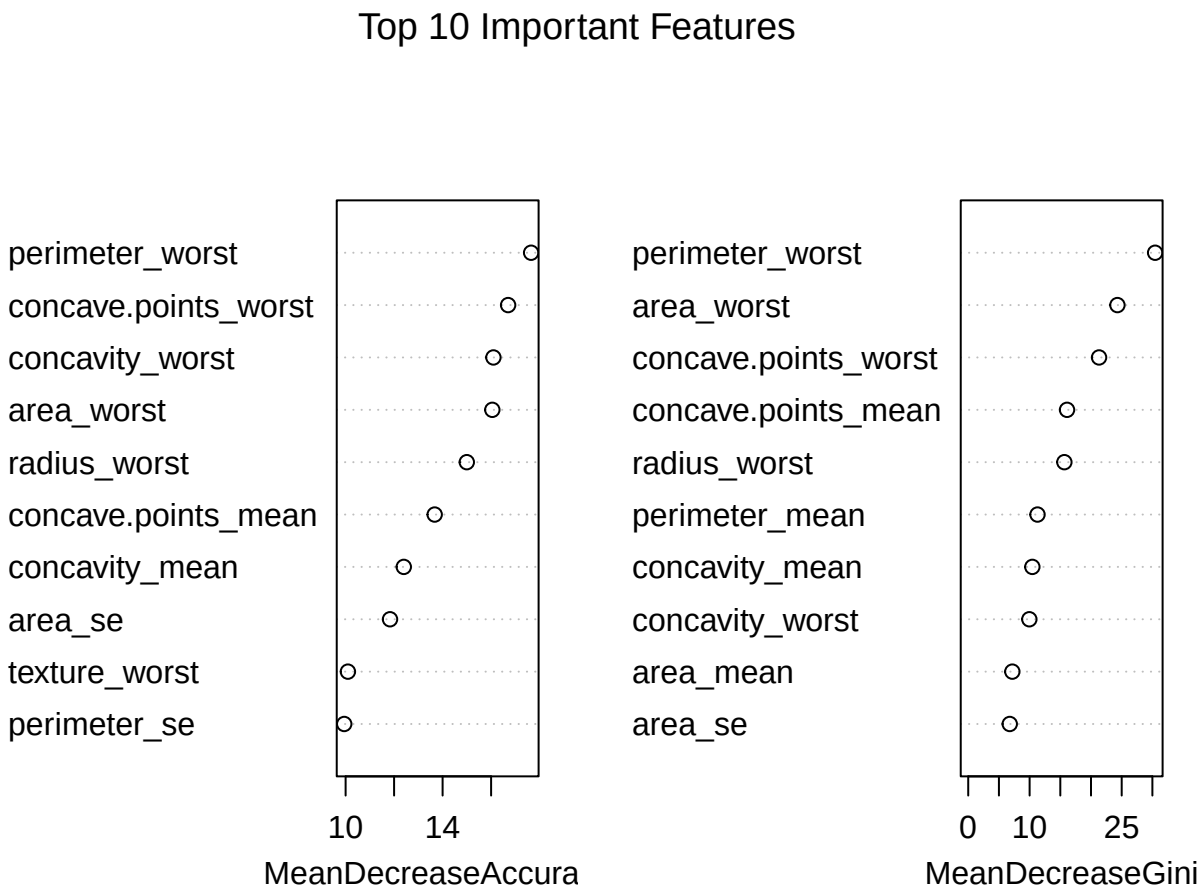
legend("bottomright",
       legend=c(
         paste("Decision Tree AUC:", round(as.numeric(auc(r1)), 3)),
         paste("Random Forest AUC:", round(as.numeric(auc(r2)), 3))
       ),
       col=c("blue", "red"), lwd=2)
```



6. Feature Importance

Random Forest estimates which features contribute most to its classification decisions.

```r
varImpPlot(rf, n.var=10,
           main="Top 10 Important Features")
```



7. Conclusion and Limitations

```r
best_model <- metrics$Model[which.max(metrics$Accuracy)]

cat("Based on the held-out test set, the model with the higher ",
    "accuracy is **", best_model, "**.\n\n", sep="")
```

Based on the held-out test set, the model with the higher accuracy is **Random Forest**.

```r
cat("Decision Tree accuracy: ",
    round(metrics$Accuracy[1] * 100, 2), "%.\n\n", sep="")
```

Decision Tree accuracy: 90.59%.

```r
cat("Random Forest accuracy: ",
    round(metrics$Accuracy[2] * 100, 2), "%.\n\n", sep="")
```

Random Forest accuracy: 95.88%.

```r
cat("The models were compared using accuracy, sensitivity, ",
    "specificity, and ROC-AUC. Results depend on the train-test ",
    "split and dataset.\n\n", sep="")
```

The models were compared using accuracy, sensitivity, specificity, and ROC-AUC. Results depend on the train-test split and dataset.

```r
cat("**Limitations:** This is a small, publicly available dataset. ",
    "Performance on this dataset does not establish clinical safety ",
    "or suitability for diagnosing patients.\n", sep="")
```

Limitations: This is a small, publicly available dataset. Performance on this dataset does not establish clinical safety or suitability for diagnosing patients.